

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и технологий
Высшая школа программной инженерии

Курсовая работа

по дисциплине

«Объектно-Ориентированное Программирование»

по теме

«Создание графического интерфейса для заданий 1-4 на языке
программирования Java»

Выполнил

студент гр. в5130904/30322

Д.А. Летоха

Руководитель

А.П. Маслаков

Санкт-Петербург
2025

Содержание

1. Цель работы	3
2. Программа работы	3
3. Результаты работы	5
4. Вывод.....	12
5. Листинг программы.....	14

1. Постановка задачи

Разработать приложение с графическим интерфейсом для заданий 1–4.

Для этого приложения должна быть реализована возможность выбора из списка любого приложения, ввод входных данных и его выполнение.

Модифицировать задания 1–4 так, чтобы весь вывод происходил в текстовых областях, защищённых от редактирования. Предусмотреть для заданий:

- 3 - выбор файлов словаря и текста для перевода, возможность ручного ввода текста;
- 4 - ввод входных данных для методов.

Блок практических заданий 1.1-1.4 призван сформировать у студента понимание особенностей хранения, умение настраивать и поддерживать данные.

2. Структура проекта

Проект разделён на две основные части 'gui' и 'labs'. Графический интерфейс реализован в части 'gui'; исполняемый код заданий в - 'labs'.

На рисунке 1 изображена схема классов проекта.



Рис.1 Схема классов

3. Лабораторная работа 1

Задача:

В компьютерной игре герой (класс Hero) может перемещаться между двумя точками (метод move) различными способами: идти пешком, ехать на лошади, лететь и т. п. Реализовать классы, позволяющие пользователю выбирать и менять в ходе выполнения программы способ перемещения героя, используя паттерн “стратегия” (strategy).

Продемонстрировать работу реализованных классов

Реализация:

1. Lab1Panel — основной GUI-компонент:
 - Отображает консоль вывода (ConsoleOutputArea).
 - Показывает выпадающий список (JComboBox) с вариантами перемещения.
 - Кнопка «Двигаться» запускает движение героя.
2. Hero — класс героя:
 - Имеет поле movementStrategy — текущая стратегия перемещения.
 - Метод move(Consumer<String> printer) вызывает move() у стратегии и выводит результат.
3. MovementStrategy — интерфейс стратегии.
4. Реализации стратегии:
 - WalkStrategy — пешком.
 - HorseRideStrategy — на лошади.
 - FlyStrategy — на грифоне.
5. MovementOption (enum) — содержит:
 - Код, описание и фабрику стратегии.
 - Метод createStrategy() возвращает соответствующую реализацию.

Интерфейс и пример работы показан на рисунке 2.

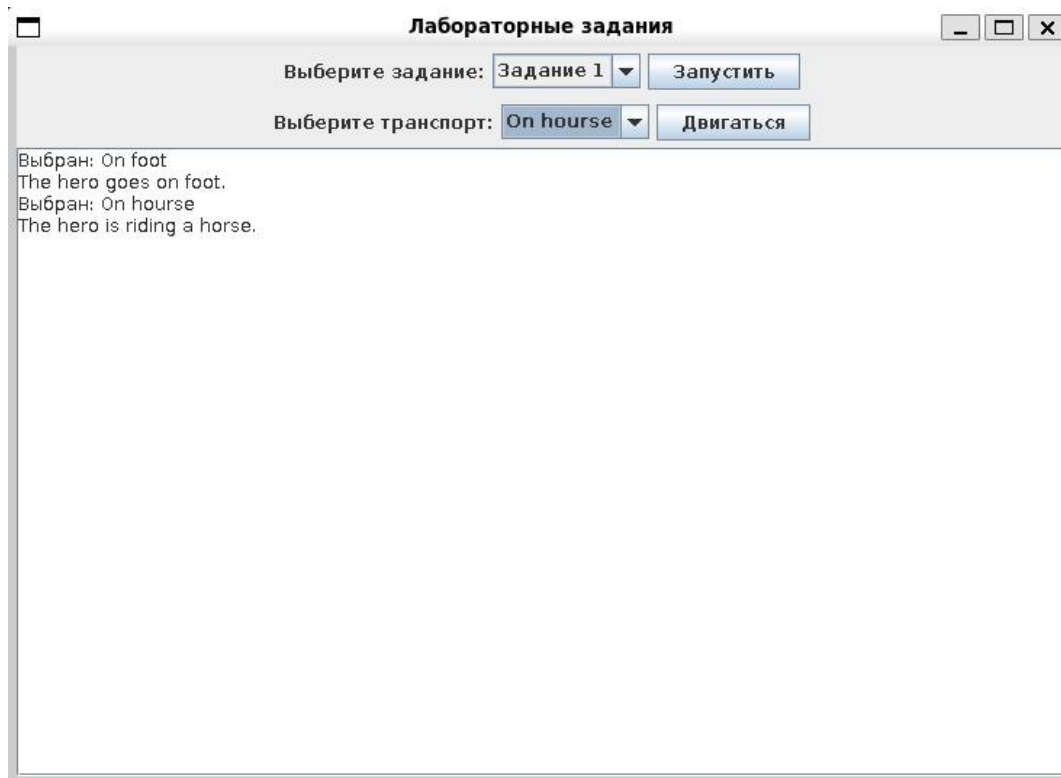


Рис.2 Интерфейс реализации задания №1

4. Лабораторная работа 2

Задача:

Написать аннотацию с целочисленным параметром. Создать класс, содержащий публичные, защищенные и приватные методы (2–3 каждого вида) с параметрами, аннотировать любые из них. Вызвать из другого класса все аннотированные защищенные и приватные методы столько раз, сколько указано в параметре аннотации. Вызывающий методы код не должен зависеть от количества и типов параметров этих методов.

Реализация:

1. Аннотация @Repeat

- Применяется к методам.
- Указывает, сколько раз метод должен быть вызван.

2. Класс MyClass

Содержит методы с аннотацией @Repeat:

- Публичные, защищённые и приватные.
- Примеры:

3. Класс AnnotationInvoker

Главный исполнитель:

- Ищет методы MyClass, помеченные @Repeat.
- Вызывает их через рефлексию нужное количество раз.
- Передаёт аргументы, введённые пользователем, и Consumer<String> для вывода.

Ключевые шаги:

1. Получает все методы класса MyClass.
2. Фильтрует защищённые и приватные методы с @Repeat.
3. Парсит входные аргументы из строки.
4. Вызывает метод repeat.value() раз.
5. Выводит результат (если метод возвращает значение).

4. GUI: Lab2Panel

- Поле ввода аргументов.
- Кнопка «Выполнить методы с @Repeat».
- Консоль вывода (ConsoleOutputArea).

Поведение:

- При нажатии кнопки:
 - Очищается консоль.
 - Ввод передаётся в AnnotationInvoker.runWithInput(...).
 - Вывод отображается в консоли.

Пример работы:

Если пользователь вводит: «Привет, 42»

и метод log(String, int, Consumer) помечен @Repeat(3), то результат будет:

«

LOG 42: Привет

LOG 42: Привет

LOG 42: Привет

»

Интерфейс и пример работы показан на рисунке 3.

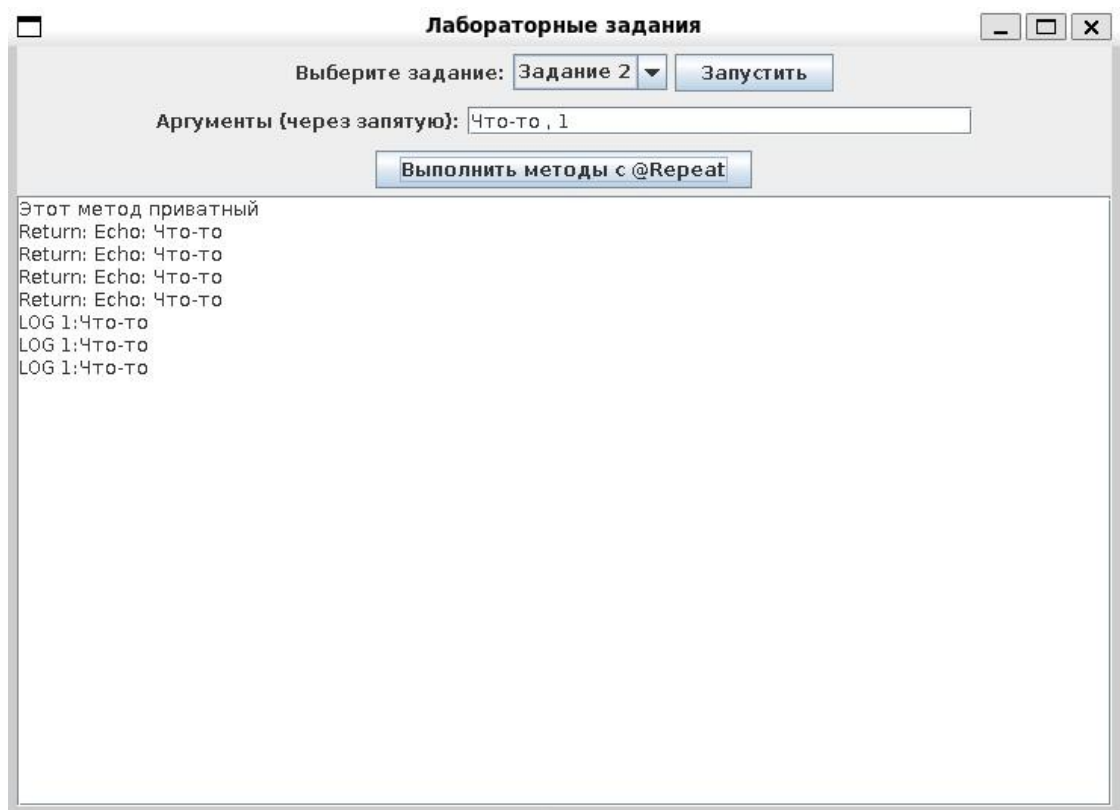


Рис.3 Интерфейс реализации задания №2

5. Лабораторная работа 3

Задача:

Реализовать программу-переводчик.

При запуске программы выполняется чтение словаря из файла, записанного в следующем формате:

слово или выражение | перевод

Затем запрашивается пользовательский ввод в консоли и выполняется перевод введенного текста.

Перевод осуществляется по следующим правилам:

- регистр букв игнорируется
- если искомого слова нет в словаре – выводится без перевода
- если есть несколько подходящих вариантов, выбирается вариант с максимальной длиной левой части.

Результат перевода выводится в консоль

Создать и применить пользовательские исключения:

- `InvalidFileFormatException`

- `FileNotFoundException` файла не существует, нет доступа к файлу и т. д.

Реализация:

Lab3Panel (GUI)

- Поле ввода текста.
- Кнопка «Загрузить словарь» — открывает диалог выбора файла.
- Кнопка «Перевести» — запускает перевод.
- Консоль вывода (`ConsoleOutputArea`).

DictionaryLoader

Загружает словарь из файла:

- Каждая строка должна содержать ключ и перевод, разделённые символом `\n`.
- Пример строки:
- Словарь сортируется по убыванию длины ключа, чтобы избежать частичных совпадений (например, "he" раньше "hello").

Обработка ошибок:

- `FileNotFoundException` — проблемы с чтением файла.
- `InvalidFileFormatException` — ошибки формата (например, отсутствует `\n`).

Translator

Метод `translate(String text, Map<String, String> dict)`:

- Проходит по всем ключам словаря.
- Использует регулярные выражения (`\b` — границы слова) для замены.
- Не учитывает регистр (`Pattern.CASE_INSENSITIVE`).

Поведение:

1. Пользователь нажимает «Загрузить словарь» → выбирает файл.
2. Файл читается и парсится в `Map<String, String>`.
3. Вводит текст → нажимает «Перевести».
4. `Translator.translate(...)` заменяет слова по словарю.

5. Результат выводится в консоль.

Интерфейс и пример работы показан на рисунке 4.

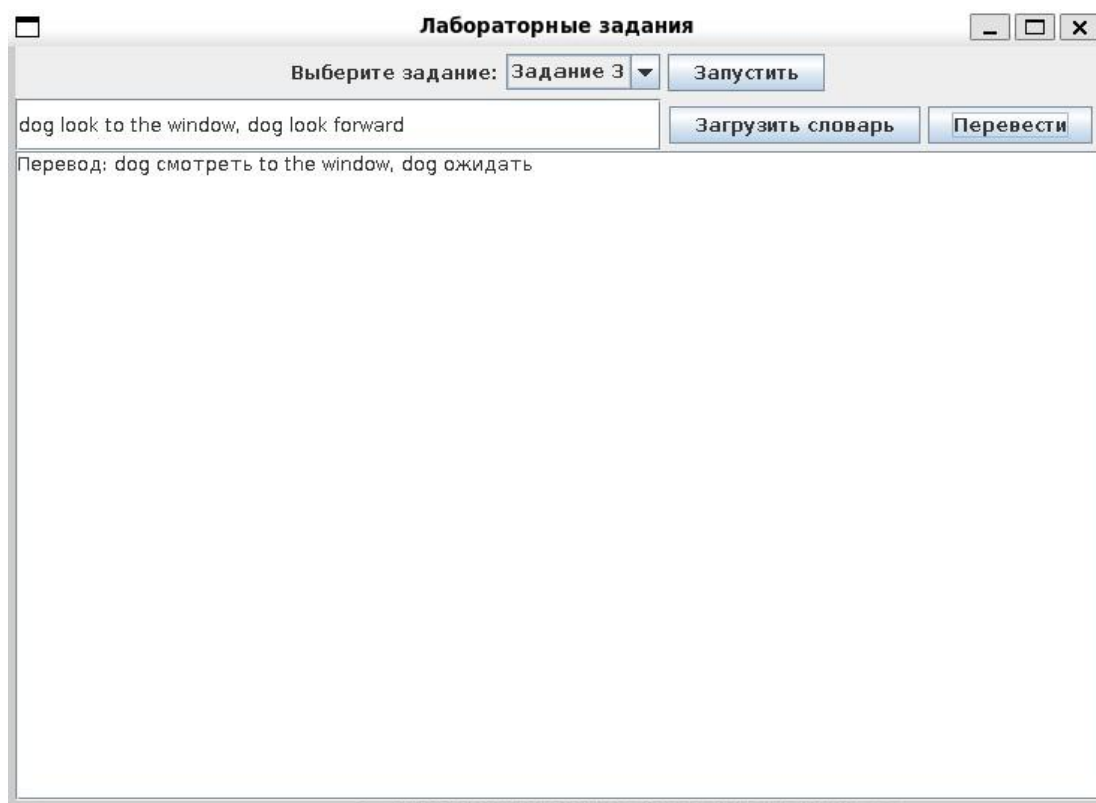


Рис.4 Интерфейс реализации задания №3

6. Лабораторная работа 4

Задача:

С использованием только Stream API реализовать следующие методы:

- метод, возвращающий среднее значение списка целых чисел;
- метод, приводящий все строки в списке в верхний регистр и добавляющий к ним префикс «_new_»;
- метод, возвращающий список квадратов всех встречающихся только один раз элементов списка;
- метод, принимающий на вход коллекцию и возвращающий ее последний элемент или кидающий исключение, если коллекция пуста;
- метод, принимающий на вход массив целых чисел, возвращающий сумму чётных чисел или 0, если чётных чисел нет;

- метод, преобразовывающий все строки в списке в Map, где первый символ – ключ, оставшиеся – значение;

Реализация:

Lab4Panel (GUI)

- Два поля ввода:
 - Числа (через запятую) — InputForm, преобразует ввод в List<Integer> и int[].
 - Строки (через запятую) — InputForm, преобразует ввод в List<String>.
- Кнопки операций:
 - Среднее
 - Сумма чётных
 - Квадраты уникальных
 - Преобразовать строки
 - Последний элемент
 - Строки в Map
- Консоль вывода (ConsoleOutputArea).

MyMethods — логика обработки

Каждая кнопка вызывает соответствующий метод:

Кнопка	Метод	Описание
Среднее	average(List<Integer>)	Вычисляет среднее значение чисел.
Сумма чётных	sumEven(int[])	Суммирует только чётные числа.
Квадраты уникальных	uniqueSquares(List<Integer>)	Возвращает квадраты чисел, которые встречаются один раз.
Преобразовать строки	transformStrings(List<String>)	Преобразует строки в верхний регистр с префиксом <code>_new_</code> .
Последний элемент	getLastElement(Collection<T>)	Возвращает последний элемент коллекции.
Строки в Map	toCharMap(List<String>)	Преобразует строки в Map<Character, String>, где ключ — первый символ, значение — остальная часть строки.

Поведение:

1. Пользователь вводит данные в поля.
2. Нажимает кнопку.
3. GUI вызывает соответствующий метод из MyMethods.
4. Результат выводится в консоль.

Интерфейс и пример работы показан на рисунке 5.

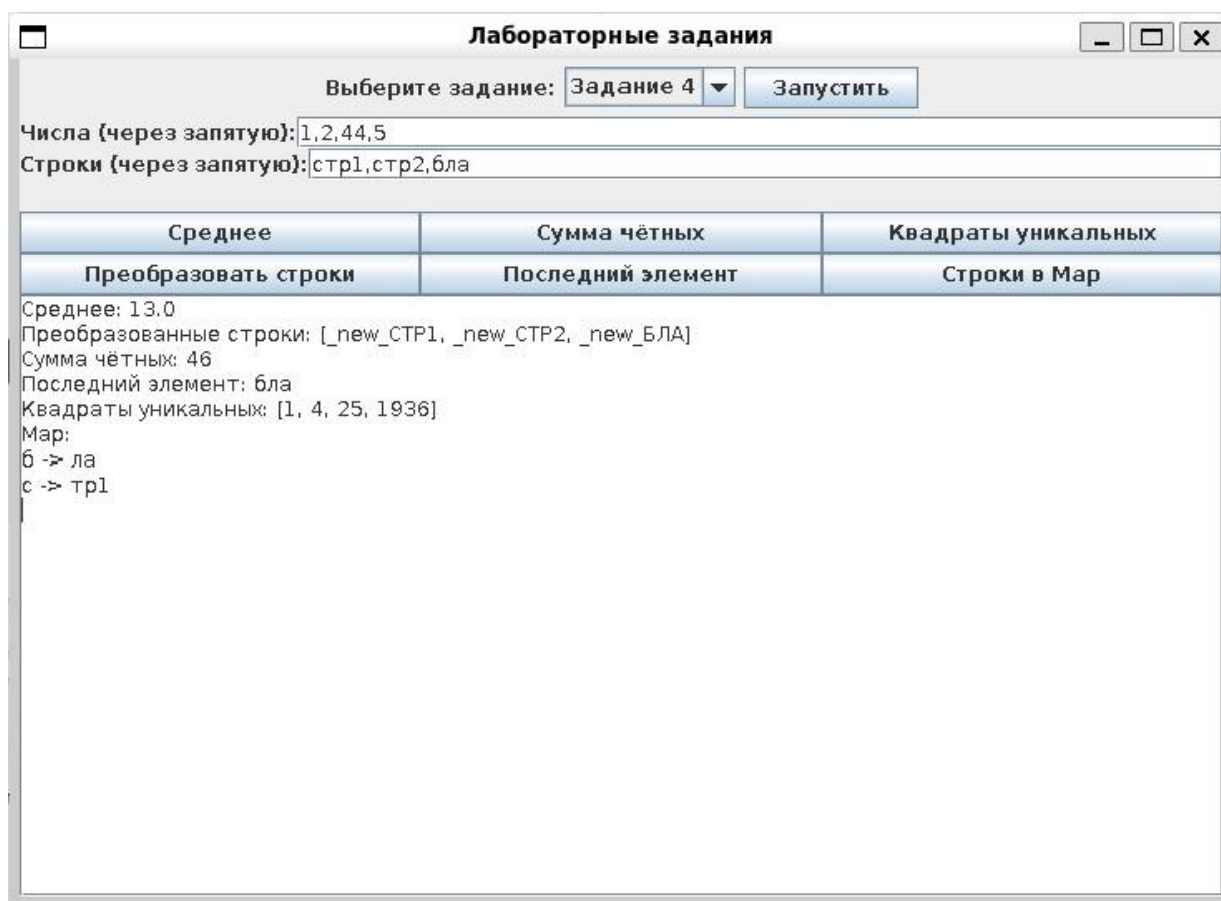


Рис.5 Интерфейс реализации задания №4

7. Вывод

В рамках курсового проекта было разработано приложение с графическим интерфейсом на языке программирования Java, с использованием Swing и среды разработки VS Code. Приложение предназначено для выполнения лабораторных работ №1–4.

Целью создания приложения было предоставление пользователю удобного и интуитивно понятного интерфейса для:

- выбора лабораторной работы;

- ввода исходных данных;
- получения результатов выполнения;
- обработки ошибок и некорректного ввода.

Программа включает:

- реализацию паттерна Стратегия (Lab1);
- работу с аннотациями и рефлексией (Lab2);
- перевод текста с использованием загружаемого словаря (Lab3);
- обработку коллекций через Stream API (Lab4).

Каждая лабораторная работа интегрирована в общее приложение и запускается через единый интерфейс. Вывод результатов осуществляется в консольную область, а ошибки отображаются в виде всплывающих сообщений.

Приложение успешно демонстрирует корректную работу всех лабораторных заданий. Практические результаты соответствуют теоретическим ожиданиям, а интерфейс обеспечивает удобство взаимодействия с пользователем.

Все поставленные цели курсового проекта были достигнуты. Программа является завершённым и функциональным решением для выполнения лабораторных работ, обеспечивая наглядность, удобство и надёжность.

8. Листинг программы

```
package gui.components;

import javax.swing.*;

public class ConsoleOutputArea extends JTextArea {
    public ConsoleOutputArea() {
        super(15, 50);
        setEditable(false);
        setLineWrap(true);
        setWrapStyleWord(true);
    }

    public void print(String text) {
        append(text + "\n");
    }

    public void clear() {
        setText("");
    }
}

package gui.components;

import javax.swing.*;
import java.awt.*;
import java.util.Arrays;
import java.util.List;

public class InputForm extends JPanel {
    private final JTextField inputField;

    public InputForm(String labelText) {
        setLayout(new BorderLayout());
        JLabel label = new JLabel(labelText);
        inputField = new JTextField();
        add(label, BorderLayout.WEST);
        add(inputField, BorderLayout.CENTER);
    }

    public List<String> getStringList() {
        String text = inputField.getText().trim();
        return Arrays.asList(text.split("\\s+", "\\s+"));
    }

    public List<Integer> getIntegerList() {
        try {
            return getStringList().stream()
                .map(Integer::parseInt)
                .toList();
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Некорректный ввод чисел",
                "Ошибка", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

```

        return List.of();
    }
}

public int[] getIntArray() {
    return getIntegerList().stream().mapToInt(i -> i).toArray();
}

public void clear() {
    inputField.setText("");
}
}

package gui;

import javax.swing.*;
import java.awt.BorderLayout;
import gui.components.ConsoleOutputArea;
import labs.lab1.game.Hero;
import labs.lab1.strategy.MovementOption;

public class Lab1Panel extends JPanel {
    public Lab1Panel() {
        setLayout(new BorderLayout());

        ConsoleOutputArea output = new ConsoleOutputArea();
        add(new JScrollPane(output), BorderLayout.CENTER);

        // Выбор стратегии из доступных
        JComboBox<MovementOption> transportBox = new
JComboBox<>(MovementOption.values());
        JButton moveButton = new JButton("Двигаться");

        Hero hero = new Hero();

        moveButton.addActionListener(e -> {
            MovementOption option = (MovementOption)
transportBox.getSelectedItem();
            if (option != null) {
                hero.setMovementStrategy(option.createStrategy());
                output.print("Выбран: " + option.getDescription());
                hero.move(output::print);
            } else {
                output.print("Стратегия не выбрана.");
            }
        });

        JPanel controlPanel = new JPanel();
        controlPanel.add(new JLabel("Выберите транспорт:"));
        controlPanel.add(transportBox);
        controlPanel.add(moveButton);
    }
}

```

```

        add(controlPanel, BorderLayout.NORTH);
    }
}

package gui;

import java.awt.*;
import javax.swing.*;
import gui.components.ConsoleOutputArea;
import labs.lab2.invoker.AnnotationInvoker;

public class Lab2Panel extends JPanel {
    public Lab2Panel() {
        setLayout(new BorderLayout());

        ConsoleOutputArea output = new ConsoleOutputArea();
        add(new JScrollPane(output), BorderLayout.CENTER);

        JTextField inputField = new JTextField(30);
        JButton runButton = new JButton("Выполнить методы с @Repeat");

        runButton.addActionListener(e -> {
            output.clear();
            String userInput = inputField.getText();
            AnnotationInvoker.runWithInput(userInput, output::print);
        });

        JPanel controlPanel = new JPanel();
        controlPanel.setLayout(new BoxLayout(controlPanel,
        BoxLayout.Y_AXIS));

        JPanel inputPanel = new JPanel(new FlowLayout(FlowLayout.CENTER));
        inputPanel.add(new JLabel("Аргументы (через запятую):"));
        inputPanel.add(inputField);

        JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER));
        buttonPanel.add(runButton);

        controlPanel.add(inputPanel);
        controlPanel.add(buttonPanel);

        add(controlPanel, BorderLayout.NORTH);
    }
}

package gui;

import java.awt.BorderLayout;
import java.io.File;
import java.util.Map;

```



```

import javax.swing.JButton;
import javax.swing.JFileChooser;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextField;

import gui.components.ConsoleOutputArea;
import labs.lab3.loader.DictionaryLoader;
import labs.lab3.service.Translator;

public class Lab3Panel extends JPanel {
    private Map<String, String> dictionary;

    public Lab3Panel() {
        setLayout(new BorderLayout());

        ConsoleOutputArea output = new ConsoleOutputArea();
        add(new JScrollPane(output), BorderLayout.CENTER);

        JTextField inputField = new JTextField();
        JButton translateButton = new JButton("Перевести");
        JButton loadDictButton = new JButton("Загрузить словарь");

        loadDictButton.addActionListener(e -> {
            JFileChooser chooser = new JFileChooser();
            int result = chooser.showOpenDialog(this);
            if (result == JFileChooser.APPROVE_OPTION) {
                File file = chooser.getSelectedFile();
                try {
                    dictionary =
DictionaryLoader.load(file.getAbsolutePath());
                } catch (Exception ex) {
                    output.print("Ошибка: " + ex.getMessage());
                }
            }
        });

        translateButton.addActionListener(e -> {
            if (dictionary == null) {
                output.print("Словарь не загружен.");
                return;
            }
            String text = inputField.getText();
            String translated = Translator.translate(text, dictionary);
            output.print("Перевод: " + translated);
        });

        JPanel topPanel = new JPanel(new BorderLayout());
        topPanel.add(inputField, BorderLayout.CENTER);
        JPanel buttons = new JPanel();
        buttons.add(loadDictButton);
        buttons.add(translateButton);
        topPanel.add(buttons, BorderLayout.EAST);
    }
}

```

```

        add(topPanel, BorderLayout.NORTH);
    }
}

```

```
package gui;
```

```

import javax.swing.*;
import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.util.List;
import java.util.Map;
import gui.components.ConsoleOutputArea;
import gui.components.InputForm;
import labs.lab4.steamApiMethods.MyMethods;

```

```

public class Lab4Panel extends JPanel {
    public Lab4Panel() {
        setLayout(new BorderLayout());

        ConsoleOutputArea output = new ConsoleOutputArea();
        add(new JScrollPane(output), BorderLayout.CENTER);

        InputForm intListForm = new InputForm("Числа (через запятую):");
        InputForm stringListForm = new InputForm("Строки (через
запятую):");

        JButton avgButton = new JButton("Среднее");
        JButton transformButton = new JButton("Преобразовать строки");
        JButton uniqueSquaresButton = new JButton("Квадраты уникальных");
        JButton lastElementButton = new JButton("Последний элемент");
        JButton evenSumButton = new JButton("Сумма чётных");
        JButton toMapButton = new JButton("Строки в Map");

        avgButton.addActionListener(e -> {
            List<Integer> nums = intListForm.getIntegerList();
            double avg = MyMethods.average(nums);
            output.print("Среднее: " + avg);
        });

        transformButton.addActionListener(e -> {
            List<String> strs = stringListForm.getStringList();
            List<String> result = MyMethods.transformStrings(strs);
            output.print("Преобразованные строки: " + result);
        });

        uniqueSquaresButton.addActionListener(e -> {
            List<Integer> nums = intListForm.getIntegerList();
            List<Integer> result = MyMethods.uniqueSquares(nums);
            output.print("Квадраты уникальных: " + result);
        });
    }
}

```

```

lastElementButton.addActionListener(e -> {
    List<String> strs = stringListForm.getStringList();
    String last = MyMethods.getLastElement(strs);
    output.print("Последний элемент: " + last);
});

evenSumButton.addActionListener(e -> {
    int[] arr = intListForm.getIntArray();
    int sum = MyMethods.sumEven(arr);
    output.print("Сумма чётных: " + sum);
});

toMapButton.addActionListener(e -> {
    List<String> strs = stringListForm.getStringList();
    Map<Character, String> map = MyMethods.toCharMap(strs);
    output.print("Map:");
    map.forEach((k, v) -> output.print(k + " -> " + v));
});

JPanel inputPanel = new JPanel(new GridLayout(3, 1));
inputPanel.add(intListForm);
inputPanel.add(stringListForm);

JPanel buttonPanel = new JPanel(new GridLayout(2, 3));
buttonPanel.add(avgButton);
buttonPanel.add(evenSumButton);
buttonPanel.add(uniqueSquaresButton);
buttonPanel.add(transformButton);
buttonPanel.add(lastElementButton);
buttonPanel.add(toMapButton);

JPanel topPanel = new JPanel(new BorderLayout());
topPanel.add(inputPanel, BorderLayout.CENTER);
topPanel.add(buttonPanel, BorderLayout.SOUTH);

add(topPanel, BorderLayout.NORTH);
}
}

```

```

package gui;

```

```

import javax.swing.*;
import java.awt.*;

```

```

public class MainApp {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Лабораторные задания");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(800, 600);

```

```

        String[] labs = { "Задание 1", "Задание 2", "Задание 3",
"Задание 4" };
        JComboBox<String> labSelector = new JComboBox<>(labs);
        JPanel contentPanel = new JPanel(new BorderLayout());

        JButton launchButton = new JButton("Запустить");
        launchButton.addActionListener(e -> {
            String selected = (String) labSelector.getSelectedItem();
            contentPanel.removeAll();
            switch (selected) {
                case "Задание 1" -> contentPanel.add(new Lab1Panel(),
BorderLayout.CENTER);
                case "Задание 2" -> contentPanel.add(new Lab2Panel(),
BorderLayout.CENTER);
                case "Задание 3" -> contentPanel.add(new Lab3Panel(),
BorderLayout.CENTER);
                case "Задание 4" -> contentPanel.add(new Lab4Panel(),
BorderLayout.CENTER);
            }
            contentPanel.revalidate();
            contentPanel.repaint();
        });

        JPanel topPanel = new JPanel();
        topPanel.add(new JLabel("Выберите задание:"));
        topPanel.add(labSelector);
        topPanel.add(launchButton);

        frame.getContentPane().add(topPanel, BorderLayout.NORTH);
        frame.getContentPane().add(contentPanel, BorderLayout.CENTER);
        frame.setVisible(true);
    });
}
}

```

```

package labs.lab1.game;

```

```

import java.util.function.Consumer;

```

```

import labs.lab1.strategy.MovementStrategy;

```

```

import labs.lab1.strategy.MovementStrategyFactory;

```

```

package labs.lab1.game;

```

```

import java.util.function.Consumer;

```

```

import labs.lab1.strategy.MovementStrategy;

```

```

public class Hero {

```

```

    private MovementStrategy movementStrategy;

```

```

    public void setMovementStrategy(MovementStrategy strategy) {

```

```

        this.movementStrategy = strategy;
    }

    public void move(Consumer<String> printer) {
        if (movementStrategy != null) {
            movementStrategy.move(printer);
        } else {
            printer.accept("No movement strategy specified");
        }
    }
}

package labs.lab1.strategy;

import java.util.function.Consumer;

public class FlyStrategy implements MovementStrategy {
    public void move(Consumer<String> printer) {
        printer.accept("The hero flies");
    }
}

package labs.lab1.strategy;

import java.util.function.Consumer;

public class HorseRideStrategy implements MovementStrategy {
    public void move(Consumer<String> printer) {
        printer.accept("The hero is riding a horse.");
    }
}

package labs.lab1.strategy;

import java.util.Arrays;
import java.util.Optional;
import java.util.function.Supplier;

public enum MovementOption {
    WALK("1", "On foot", WalkStrategy::new),
    HORSE("2", "On horse", HorseRideStrategy::new),
    FLY("3", "On griffin", FlyStrategy::new);

    private final String code;
    private final String description;
    private final Supplier<MovementStrategy> supplier;

    MovementOption(String code, String description,
Supplier<MovementStrategy> supplier) {
        this.code = code;
        this.description = description;
        this.supplier = supplier;
    }
}

```

```

    public String getCode() {
        return code;
    }

    public String getDescription() {
        return description;
    }

    public MovementStrategy createStrategy() {
        return supplier.get();
    }

    public static Optional<MovementOption> fromCode(String code) {
        return Arrays.stream(values())
            .filter(opt -> opt.code.equals(code))
            .findFirst();
    }

    @Override
    public String toString() {
        return getDescription();
    }
}

package labs.lab1.strategy;

import java.util.function.Consumer;

public interface MovementStrategy {
    void move(Consumer<String> printer);
}

package labs.lab1.strategy;

public class MovementStrategyFactory {

    // Возвращает новую стратегию по коду, или null, если код неизвестен.

    public static MovementStrategy create(String code) {
        return MovementOption
            .fromCode(code)
            .map(MovementOption::createStrategy)
            .orElse(null);
    }
}

package labs.lab1.strategy;

import java.util.function.Consumer;

public class WalkStrategy implements MovementStrategy {

```

```

        public void move(Consumer<String> printer) {
            printer.accept("The hero goes on foot.");
        }
    }
}

```

```

package labs.lab2.annotaton;

```

```

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;

```

```

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Repeat {
    int value();
}

```

```

package labs.lab2.core;

```

```

import labs.lab2.annotaton.Repeat;
import java.util.function.Consumer;

```

```

public class MyClass {
    // Public-методы
    @Repeat(2)
    public void greet(String name, Consumer<String> printer) {
        printer.accept("Hello, " + name);
    }

    public int sum(int a, int b) {
        return a + b;
    }

    // Protected-методы
    @Repeat(3)
    protected void log(String message, int num, Consumer<String> printer) {
        printer.accept("LOG " + num + ":" + message);
    }

    protected double multiply(double x, double y) {
        return x * y;
    }

    // Private-методы
    @Repeat(1)
    private void secret(Consumer<String> printer) {
        printer.accept("Этот метод приватный");
    }

    @Repeat(4)
    private String echo(String s) {
        return "Echo: " + s;
    }
}

```

```
}  
}
```

```
package labs.lab2.invoker;
```

```
import java.lang.reflect.Method;  
import java.lang.reflect.Modifier;  
import java.util.function.Consumer;  
import labs.lab2.annotaton.Repeat;  
import labs.lab2.core.MyClass;
```

```
public class AnnotationInvoker {
```

```
    public static void runWithInput(String input, Consumer<String>  
outputConsumer) {  
        MyClass target = new MyClass();  
        Class<?> cls = target.getClass();  
  
        for (Method method : cls.getDeclaredMethods()) {  
            int mods = method.getModifiers();  
            if ((Modifier.isProtected(mods) || Modifier.isPrivate(mods)) &&  
                method.isAnnotationPresent(Repeat.class)) {  
  
                Repeat rep = method.getAnnotation(Repeat.class);  
                int times = rep.value();  
                method.setAccessible(true);  
  
                Class<?>[] types = method.getParameterTypes();  
                Object[] params = parseInput(input, types, outputConsumer);  
  
                try {  
                    for (int i = 0; i < times; i++) {  
                        Object result = method.invoke(target, params);  
                        if (method.getReturnType() != Void.TYPE) {  
                            outputConsumer.accept("Return: " + result);  
                        }  
                    }  
                } catch (Exception e) {  
                    outputConsumer.accept("Ошибка при вызове метода: " +  
method.getName());  
                    outputConsumer.accept("Причина: " + e.getMessage());  
                }  
            }  
        }  
    }  
}
```

```
    private static Object[] parseInput(String input, Class<?>[] types,  
Consumer<String> outputConsumer) {  
        Object[] result = new Object[types.length];  
  
        if (input == null || input.trim().isEmpty()) {  
            for (int i = 0; i < types.length; i++) {
```



```

        result[i] = defaultValue(types[i], outputConsumer);
    }
    return result;
}

String[] parts = input.split(",");
for (int i = 0; i < types.length; i++) {
    if (i < parts.length) {
        result[i] = safeConvert(parts[i].trim(), types[i],
outputConsumer);
    } else {
        result[i] = defaultValue(types[i], outputConsumer);
    }
}
return result;
}

private static Object safeConvert(String value, Class<?> type,
Consumer<String> printer) {
    try {
        return convert(value, type);
    } catch (Exception e) {
        return defaultValue(type, printer);
    }
}

private static Object convert(String value, Class<?> type) {
    if (type == int.class || type == Integer.class)
        return Integer.parseInt(value);
    if (type == double.class || type == Double.class)
        return Double.parseDouble(value);
    if (type == boolean.class || type == Boolean.class)
        return Boolean.parseBoolean(value);
    if (type == long.class || type == Long.class)
        return Long.parseLong(value);
    if (type == float.class || type == Float.class)
        return Float.parseFloat(value);
    if (type == short.class || type == Short.class)
        return Short.parseShort(value);
    if (type == byte.class || type == Byte.class)
        return Byte.parseByte(value);
    if (type == char.class || type == Character.class)
        return value.charAt(0);
    if (type == String.class)
        return value;
    throw new IllegalArgumentException("Неподдерживаемый тип: " +
type);
}

private static Object defaultValue(Class<?> type, Consumer<String>
printer) {
    if (Consumer.class.isAssignableFrom(type)) {
        return printer;
    }
}

```

```

    }
    if (!type.isPrimitive())
        return null;
    if (type == boolean.class)
        return false;
    if (type == byte.class)
        return (byte) 0;
    if (type == short.class)
        return (short) 0;
    if (type == int.class)
        return 0;
    if (type == long.class)
        return 0L;
    if (type == float.class)
        return 0f;
    if (type == double.class)
        return 0d;
    if (type == char.class)
        return '\u0000';
    return null;
}

}

package labs.lab3.exceptions;

// Выбрасывается при проблемах открытия/чтения файла
public class FileReadException extends Exception {
    public FileReadException(String message, Throwable cause) {
        super(message, cause);
    }
}

package labs.lab3.exceptions;

// Выбрасывается при ошибках формата строк в словаре
public class InvalidFileFormatException extends Exception {
    public InvalidFileFormatException(String message) {
        super(message);
    }
}

package labs.lab3.loader;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import labs.lab3.exceptions.FileReadException;

```

```

import labs.lab3.exceptions.InvalidFileFormatException;

public class DictionaryLoader {

    // Загружает словарь из файла, сортирует по убыванию длины ключа.

    public static Map<String, String> load(String filePath)
        throws InvalidFileFormatException, FileReaderException {
        Map<String, String> temp = new LinkedHashMap<>();
        try (BufferedReader reader = new BufferedReader(new
FileReader(filePath))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.trim().isEmpty()) {
                    continue;
                }
                int delim = line.indexOf('|');
                if (delim < 0) {
                    throw new InvalidFileFormatException("В строке
отсутствует разделитель '|': " + line);
                }
                String key = line.substring(0, delim).trim();
                String value = line.substring(delim + 1).trim();
                if (key.isEmpty() || value.isEmpty()) {
                    throw new InvalidFileFormatException("Пустой ключ или
перевод: " + line);
                }
                temp.put(key, value);
            }
        } catch (IOException e) {
            throw new FileReaderException("Не удалось прочитать файл: " +
filePath, e);
        }

        // Сортировка по длине ключа (убывание)
        List<Map.Entry<String, String>> entries = new
ArrayList<>(temp.entrySet());
        entries.sort((e1, e2) -> Integer.compare(e2.getKey().length(),
e1.getKey().length()));

        LinkedHashMap<String, String> sorted = new LinkedHashMap<>();
        for (Map.Entry<String, String> e : entries) {
            sorted.put(e.getKey(), e.getValue());
        }
        return sorted;
    }
}

package labs.lab3.service;

import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

```

```

public class Translator {

    public static String translate(String text, Map<String, String> dict) {
        String result = text;
        for (Map.Entry<String, String> entry : dict.entrySet()) {
            String key = entry.getKey();
            String translation = entry.getValue();

            Pattern pattern = Pattern.compile("\\b" + Pattern.quote(key) +
"\\b", Pattern.CASE_INSENSITIVE);
            Matcher matcher = pattern.matcher(result);
            result =
matcher.replaceAll(Matcher.quoteReplacement(translation));
        }
        return result;
    }
}

```

```

package labs.lab4.steamApiMethods;

```

```

import java.util.Arrays;
import java.util.Collection;
import java.util.List;
import java.util.Map;
import java.util.NoSuchElementException;
import java.util.function.Function;
import java.util.stream.Collectors;

```

```

public class MyMethods {
    // Среднее значение списка целых чисел
    public static double average(List<Integer> numbers) {
        return numbers.stream()
            .mapToInt(Integer::intValue)
            .average()
            .orElseThrow(() -> new IllegalArgumentException("Список
пуст"));
    }
}

```

```

// Преобразование строк в верхний регистр с префиксом "_new_"
public static List<String> transformStrings(List<String> strings) {
    return strings.stream()
        .map(s -> "_new_" + s.toUpperCase())
        .collect(Collectors.toList());
}

```

```

// Список квадратов элементов, встречающихся только один раз
public static List<Integer> uniqueSquares(List<Integer> numbers) {
    return numbers.stream()
        .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
        .entrySet().stream()
        .filter(entry -> entry.getValue() == 1)

```

```

        .map(entry -> entry.getKey() * entry.getKey())
        .collect(Collectors.toList());
    }

    // Последний элемент коллекции или исключение
    public static <T> T getLastElement(Collection<T> collection) {
        return collection.stream()
            .reduce((first, second) -> second)
            .orElseThrow(() -> new NoSuchElementException("Коллекция
пуста"));
    }

    // Сумма чётных чисел массива
    public static int sumEven(int[] array) {
        return Arrays.stream(array)
            .filter(n -> n % 2 == 0)
            .sum();
    }

    // Преобразование строк в Map: первый символ — ключ, остальные —
значение
    public static Map<Character, String> toCharMap(List<String> strings) {
        return strings.stream()
            .filter(s -> s.length() > 1)
            .collect(Collectors.toMap(
                s -> s.charAt(0),
                s -> s.substring(1),
                (v1, v2) -> v1 // если ключ повторяется, то
оставляем первый
            ));
    }
}

```